

Generador de recetas a partir de ingredientes disponibles

Aplicación Streamlit conectada a un LLM vía API, con salida JSON validable y renderizada en componentes visuales.



Cuál era el reto: Un generador de recetas a partir de ingredientes disponibles

Nuestro objetivo es que, con un solo input de ingredients sueltos, y a través de una IA generativa, poder generar una receta estructurada.

1 • Streamlit

Frontend obligatorio para prototipar aplicaciones de IA/ML con rapidez.

2 • LLM vía API

El modelo generativo se consulta desde la aplicación, no se entrena desde cero.

3 • Output parseado

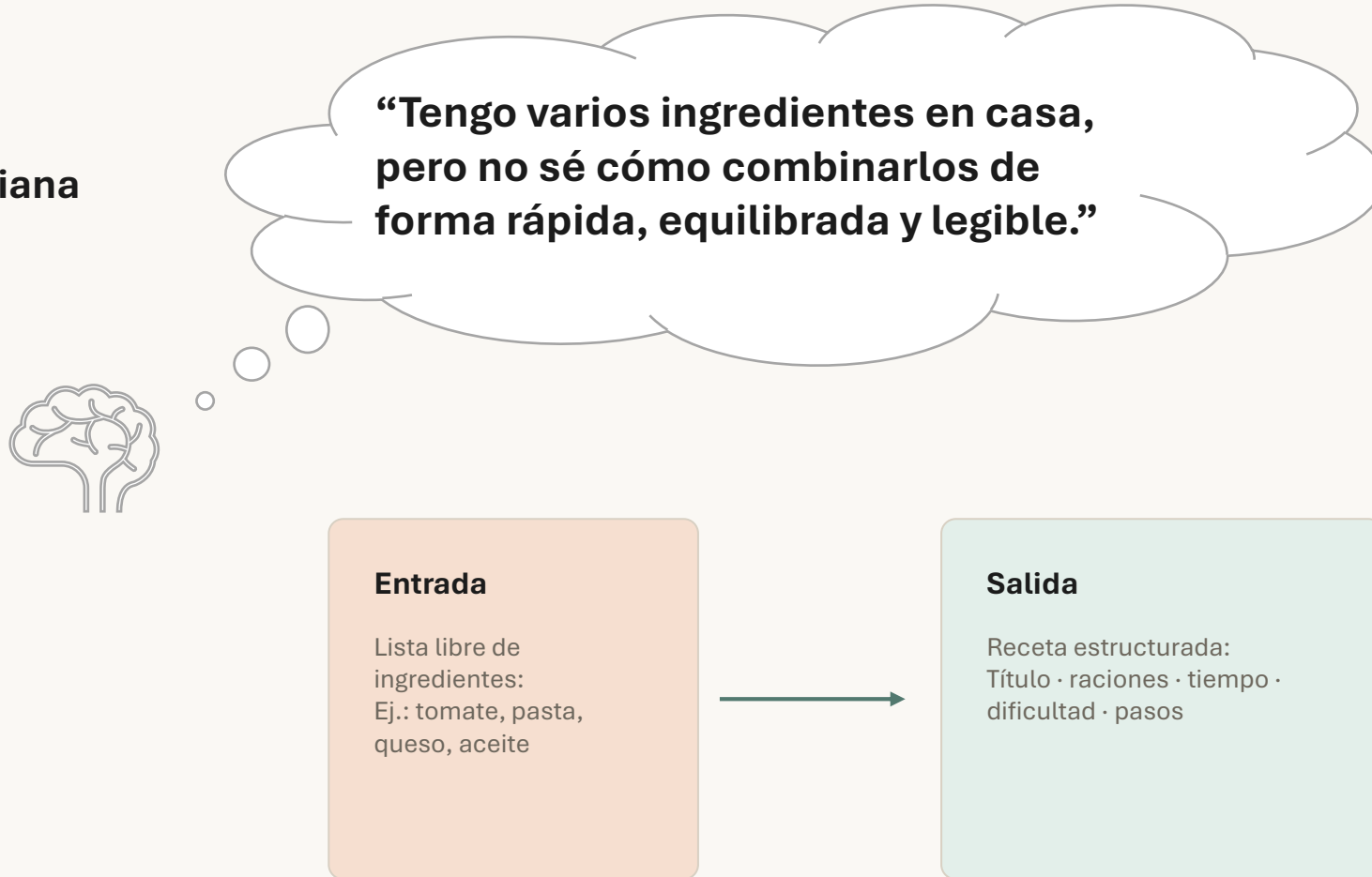
La respuesta debe mapearse a componentes de la UI: título, pasos, ingredientes...

4 • Prompt engineering

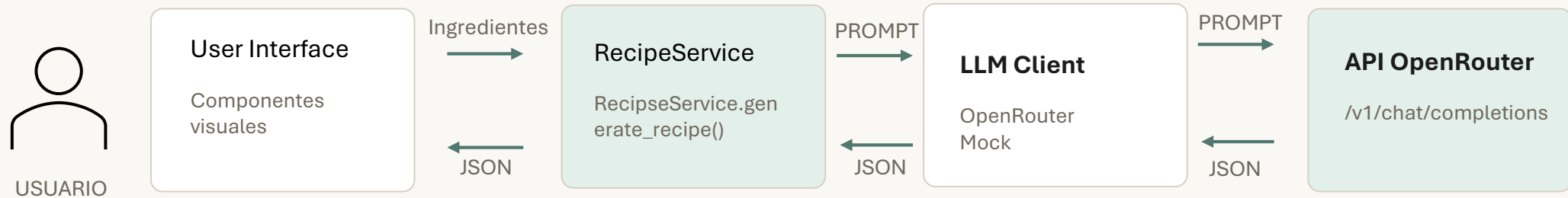
Hay que justificar técnicas: rol, contexto, formato, restricciones y parámetros.

Problema: tener ingredientes no equivale a tener una receta

Situación cotidiana



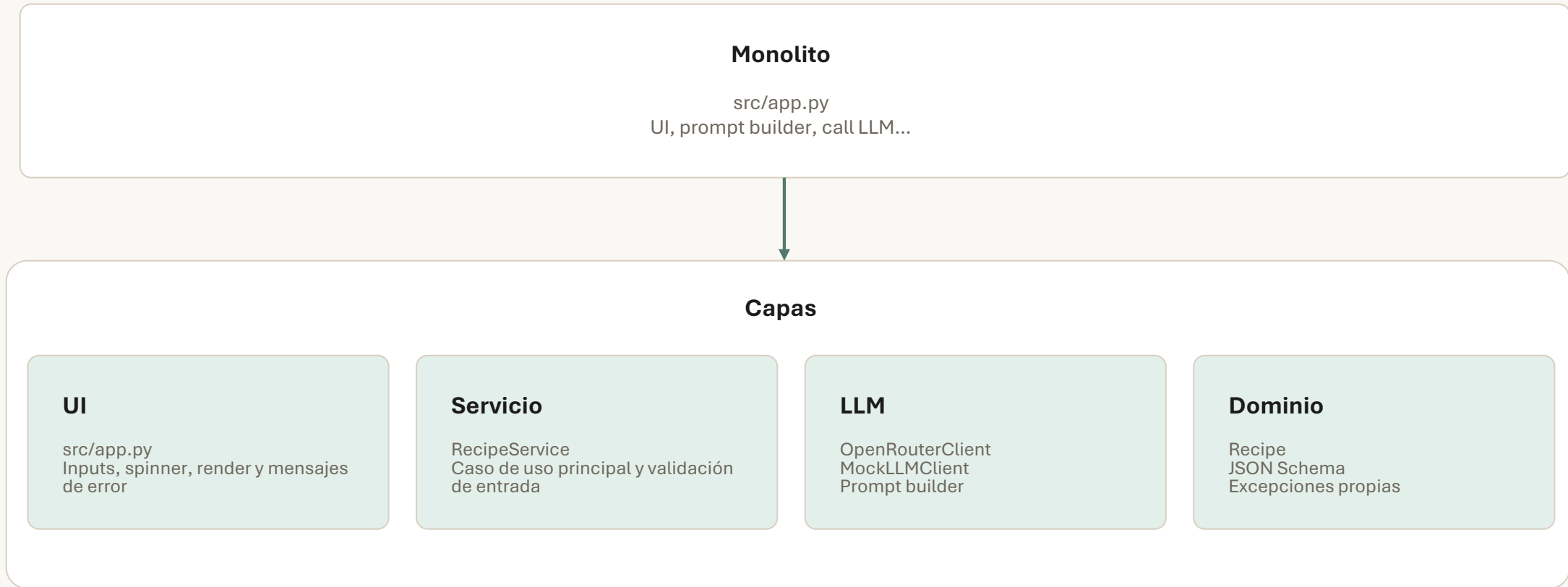
Solución propuesta



Decisión clave

El LLM genera contenido, pero la aplicación conserva el control del contrato: valida, transforma y renderiza.

Arquitectura por capas



Refactorización: de un app.py monolítico a una estructura modular en src/

Efecto: pruebas unitarias más simples, proveedor intercambiable y frontend menos acoplado a la IA.

Prompt engineering aplicado

<role>

Chef profesional, tono claro y usuario principiante.

<instructions>

Una receta, ingredientes principales, chain-of-thought

<json_schema>

Campos, sus descripciones, requisitos, tipos, títulos, etc...

<example>

Un ejemplo de salida.

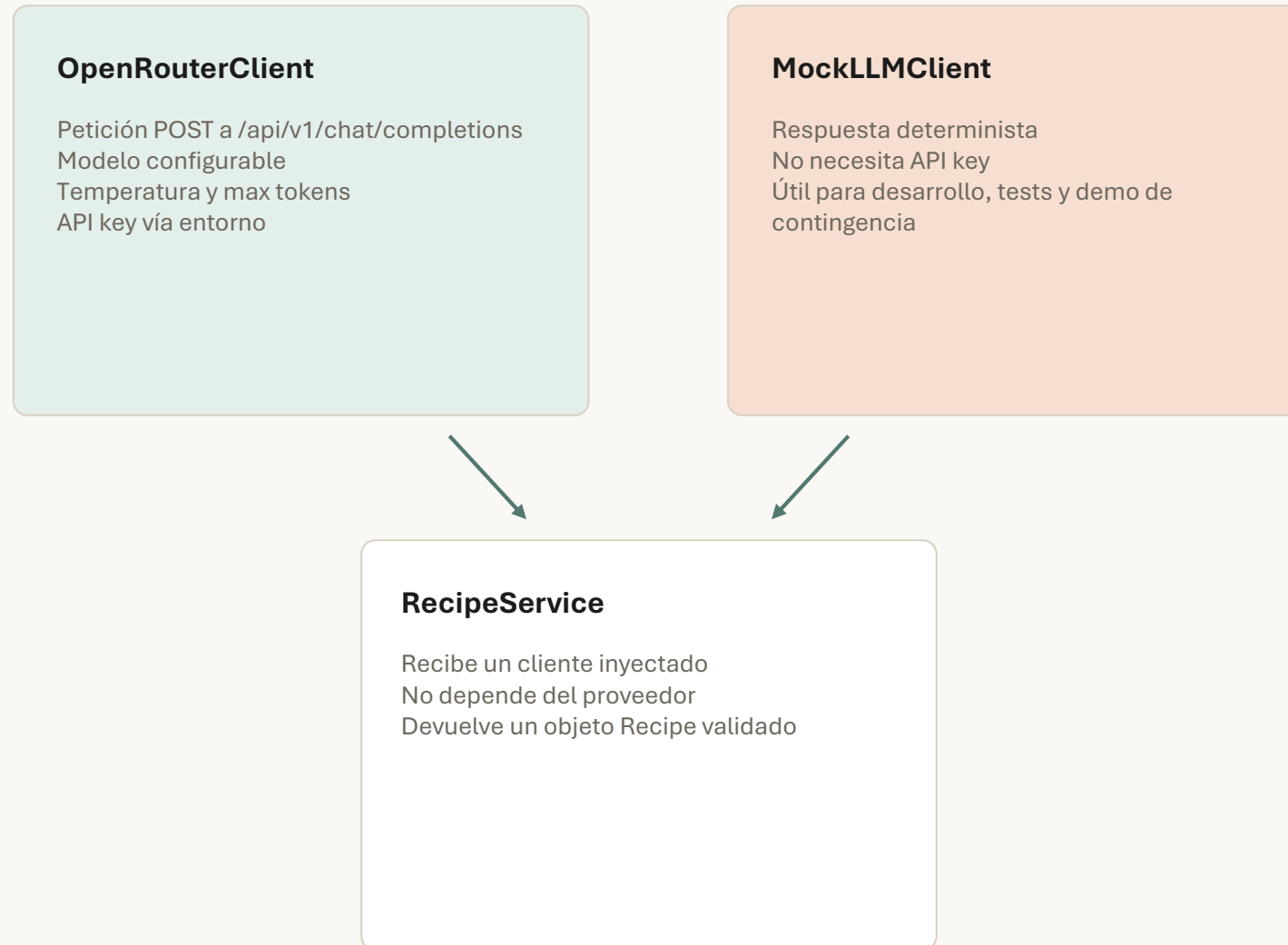
<rules>

Sin texto adicional. Sin comentarios. Solo JSON válido, etc..

```
{ "title": "...", "servings": 2, "ingredients": [...], "steps": [...] }
```

Objetivo: reducir alucinaciones de formato y evitar postprocesados frágiles.

Integración con LLM



No basta con que el modelo responda

Validación, errores y pruebas

Validación de entrada

Rechazo de ingredientes vacíos antes de llamar al LLM.

Validación de salida

Parseo JSON y comprobación contra `recipe_output.schema.json`.

Errores controlados

Configuración, petición HTTP, respuesta inválida y diagnóstico.

13 tests unitarios

Config · Mock client · Prompt builder · Recipe schema · RecipeService

Comando de ejecución: `pytest -q`

Limitaciones y líneas de trabajo a futuro

Limitaciones	Líneas de trabajo
Cuotas y límites de uso A pesar de que OpenRouter tiene hasta 26 modelos de uso gratuito, su consume está supeditado a la disponibilidad y al uso razonable.	Dependencia de LLM externo Una próxima línea de trabajo puede ser incorporar modelos locales que utilicen la CPU/GPU para generar el contenido sin depender de terceros.
Gestión de errores Ahora mismo la gestion de errores es limitada y, aunque muestra el código, no muestra la descripción del error para transparencia del usuario	Gestión de errores Distinguir aquellos errores que se deben informar al usuario para traducirlos (429 es too many requests) de aquellos errores puramente técnicos, para los cuales se debe mostrar una descripción más genérica
Seguridad La app todavía es vulnerable al prompt injection, abuso de API a un ataque DDoS.	Prompt Injection o inusuales Implementar cuotas de uso o de concurrencia de peticiones y tratar de analizar la petición para detector palabras ajenas al propósito de la APP.

Conclusión

Resultado

Un chef IA accesible desde navegador que transforma ingredientes en recetas estructuradas.

Aprendizaje

Uso práctico de modelos fundacionales, prompt engineering, parámetros de inferencia y salidas estructuradas.

Evolución

Preferencias dietéticas, seguridad alimentaria básica, varios modelos y control responsable.

Idea final

La calidad no está solo en llamar a una IA, sino en diseñar el contrato, validar la respuesta y convertirla en una experiencia útil.

¿Preguntas?

Enviar